USER ACCEPTANCE TESTING (UAT) UAT is a dedicated environment many organizations use to allow end users to test a new version of an application before it goes into production.

Automation is the key to an efficient deployment process. Any automated steps in this process will be faster and more reliable than the manual equivalents. Indeed, fast deployment relies on minimizing the number and length of manual steps in the build/release cycle.

Manual testing is usually the largest and slowest of these steps, so test automation can have a significant impact on the time required for manual testing. The automated acceptance criteria and living documentation produced by BDD also help build up confidence in the quality of the application, letting testers focus on exploratory testing and spend less time on repetitive scenario-based testing.

In this chapter, we'll take a closer look at the role BDD plays in the overall build and deployment lifecycle and how it can help streamline the automated delivery process.

12.1 Executable specifications should be part of an automated build

The executable specifications we've seen in the previous chapters, both for acceptance criteria and for lower-level technical specifications, aren't designed to be run by hand on an ad hoc basis. Although it's certainly convenient to be able to run individual tests from within an IDE, this isn't their main purpose. Rather, they're intended to be run automatically, as part of an automated build process. Individual acceptance criteria scenarios give a partial view of an application's behavior. Only when they're run together, as a comprehensive suite, can they give a full picture of the current state of the application (see figure 12.1).

For this reason, executable specifications need to work well in the context of an automated build. Whether they're implemented as low-level unit tests or end-to-end functional tests, they should respect a certain number of constraints. In particular,

- Each executable specification should be self-sufficient.
- Executable specifications should be stored in version control.
- You should be able to run the executable specifications from the command line, typically using a build script.

Let's look at these constraints in more detail.

12.1.1 Each specification should be self-sufficient

Some teams try to organize their executable specifications into *suites* that need to be executed in a precise order. For example, an initial acceptance criterion might describe how to look up an item in an online catalog, a second might then illustrate how to purchase this item, and a third might illustrate requesting a refund of this same item.

This is generally not a good idea. Executable specifications shouldn't depend on other specifications to prepare test data or to place the system in a particular state.